

AN EXPERIMENTAL COMPARISON ON A REAL-WORLD GRAPH DATASET

PostgreSQL vs Neo4j

When the graph beats the table — and when it doesn't

Giuseppe D'Angelica github.com/g1up3

Nicolas Leone github.com/theunick

Data Management 2025/26 — Sapienza Università di Roma

Tutor: Roberto Maria Delfino

No absolute winner: Neo4j dominates variable-depth traversals (55×), Postgres keeps the fixed-depth join

55×

faster shortest path on Neo4j

Q10: 792 ms in Postgres, 14 ms in Neo4j
— identical semantics, a change of complexity class.

2.7×

Postgres on the fixed-depth join (Q09)

With the same precomputation, B-tree joins beat the traversal 2.7× — and the lead widens as data grows.

4×

shorter code in Cypher

21 SQL lines become 5 Cypher lines on the shortest path; 2× shorter on average across the 12.

When does a graph database actually beat a relational one?

Same data, same questions, two paradigms — measured, not argued.

Relational strengths

- Mature planner, decades of optimisation
- B-tree indexes, OLAP-style aggregations
- Joins as a first-class, well-tuned citizen

Graph strengths

- Native traversal of relationships — no joins
- Built for paths, neighbourhoods, communities
- Schema-on-read flexibility

A real, non-trivial dataset: 25,979 matches and 917,815 events parsed from raw XML

25,979

matches

11 leagues · 8 seasons

11,060

players

with FIFA-rating snapshots

917,815

events

parsed from raw XML

542,281

lineup rows

11+11 per match

SOURCE A single 300-MB monolithic SQLite file

CHALLENGE The Match table: 115 columns, XML-encoded events

From 115 columns to 4 tables: the normalisation that makes the comparison fair

BEFORE — SOURCE

115 columns

22 player ids + 44 pitch coordinates · 8
raw XML fields · 30 betting odds · 11
base fields — all in one Match table



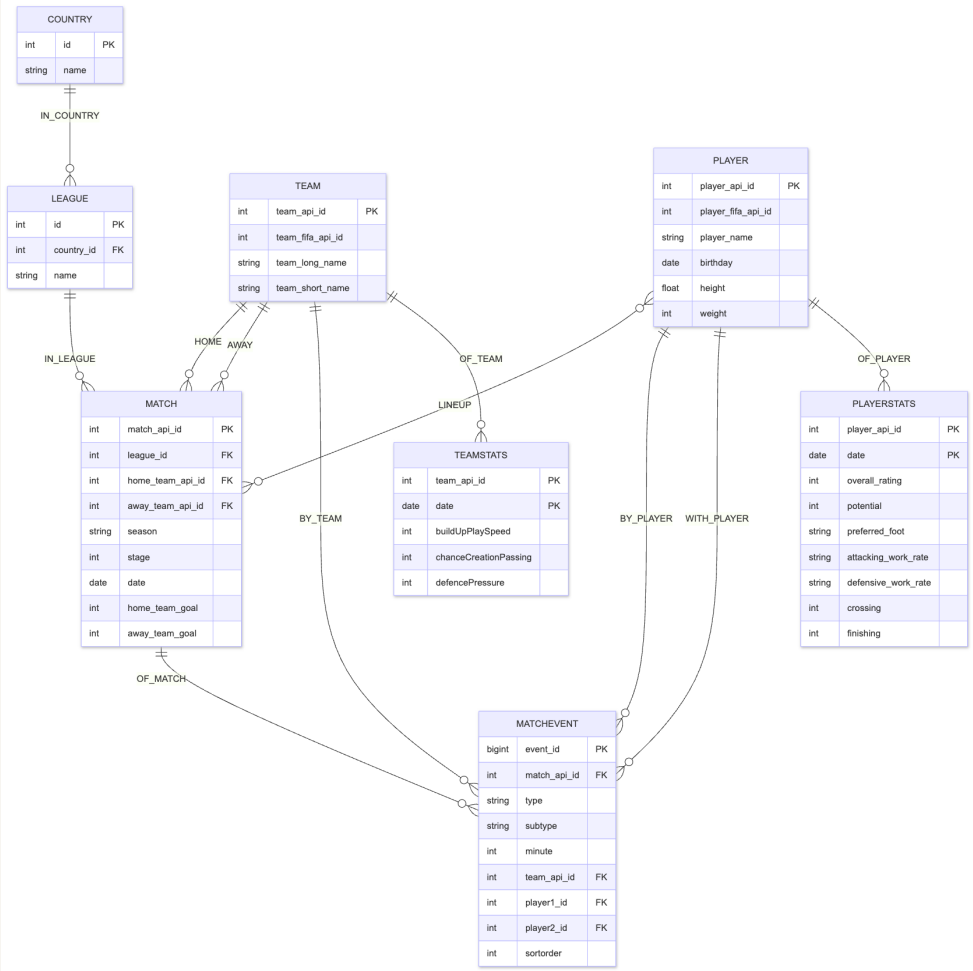
AFTER — NORMALISED

match25,979 rows

match_lineup542,281 rows

match_event917,815 rows

match_oddsout of scope



Conceptual ER — shared by both designs

1 • Readable SQL

2 • Events become indexable rows

3 • Symmetry with the graph model

In the graph, events become relationships: zero joins to navigate them

(Player) - [:SCORED_IN] -> (Match)

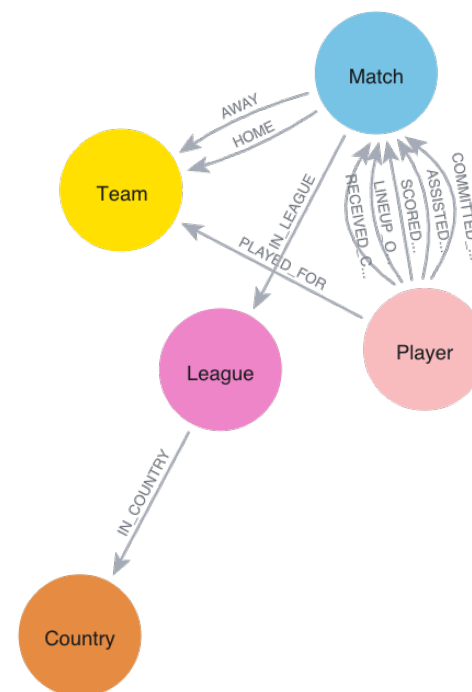
Every parsed event is a typed, directed relationship

(Player) - [:PLAYED_FOR] -> (Team)

Derived at load time — the edge that powers teammates & shortest-path queries

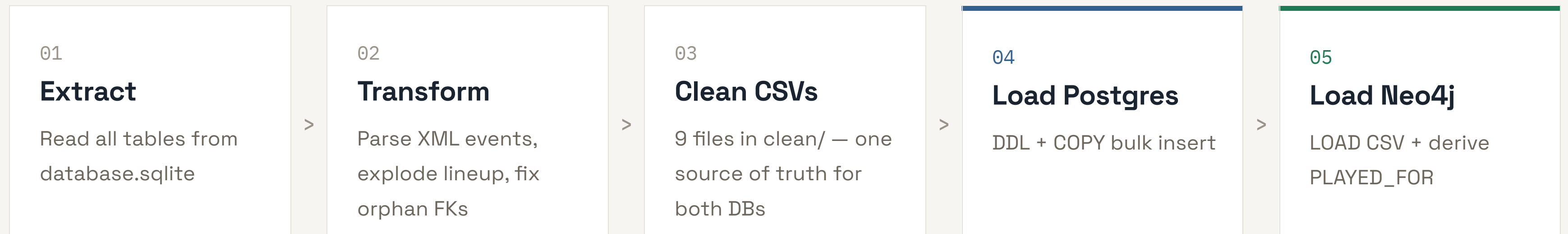
(Match) - [:HOME | AWAY] -> (Team)

Structure (league, country, sides) mirrors the relational FKs 1:1



Actual Neo4j schema visualisation — CALL
db.schema.visualization()

A reproducible pipeline: SQLite to both databases in ~7 minutes, zero orphan FKs



100% FK integrity — zero orphans across 542k+ refs

917,815 typed rows from XML (14,196 matches)

~7 min full pipeline re-run, end-to-end

Rigorous by design: 15 runs, bootstrap CIs, significance tests — and results verified equivalent

ENVIRONMENT

MacBook Air M2 · 8 GB RAM · SSD — single-user, in-memory dataset. PostgreSQL 18.6 (default config) · Neo4j 2026.04.0 Enterprise

STATISTICS

1 warm-up → 15 runs → median + 95% CI →
Mann-Whitney U

RESULT EQUIVALENCE

Both result-sets normalised (rounding, dates, decimals) and compared as multisets. **All 10 read queries: identical answers.**

FAIRNESS CHOICE

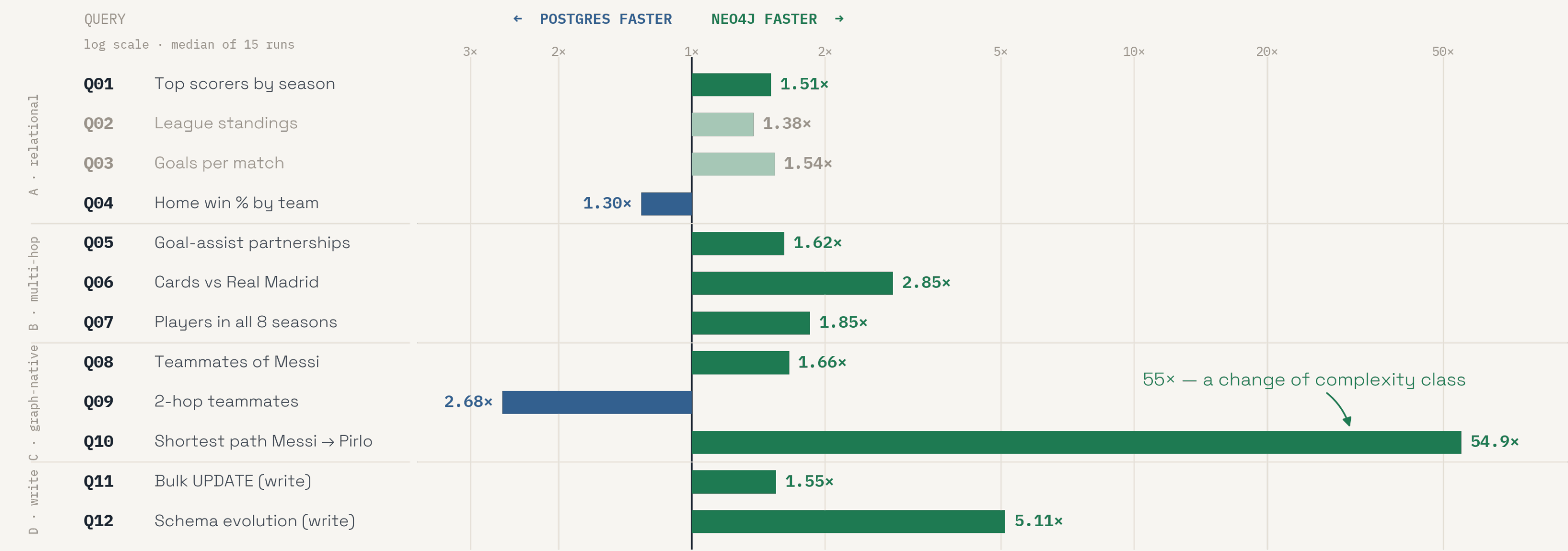
Postgres precomputes `mv_played_for` to mirror Neo4j's `PLAYED_FOR` edge — **neither system gets a structural advantage** on Q09–Q10.

12 queries of increasing graph-friendliness — with hypotheses declared before measuring

A	Q01–Q04	Aggregations & 2–3 way joins — top scorers, standings, win %	Hypothesis: Postgres wins or ties
B	Q05–Q07	Multi-hop joins, self-joins, presence-in-all-of (8 seasons)	Hypothesis: slight Neo4j edge
C	Q08–Q10	Network traversal — teammates, 2-hop, shortest path	Hypothesis: Neo4j wins clearly
D	Q11–Q12	Writes — bulk UPDATE, schema evolution (new attribute)	Comparing paradigms, not speed

Every query written twice — same logic in SQL and Cypher, parameterised · `queries/sql/Q01..Q12.sql` · `queries/cypher/Q01..Q12.cypher`

Category A is a dead heat across runs; Postgres wins the fixed-depth join, Neo4j every variable-depth traversal and every write



Postgres wins (2) Neo4j wins (8) · not significant (2) saturated bars = significant after Holm correction (10/12) · Mann-Whitney U, $p < 0.05$ · all results equivalent (multiset)

Q10 – SHORTEST PATH, MESSI → PIRLO

55x

faster on Neo4j — a change of complexity class, not an optimisation

WORK DONE BY THE ENGINE – FROM THE CAPTURED QUERY PLANS
~3 orders of magnitude less work — identical semantics

POSTGRES

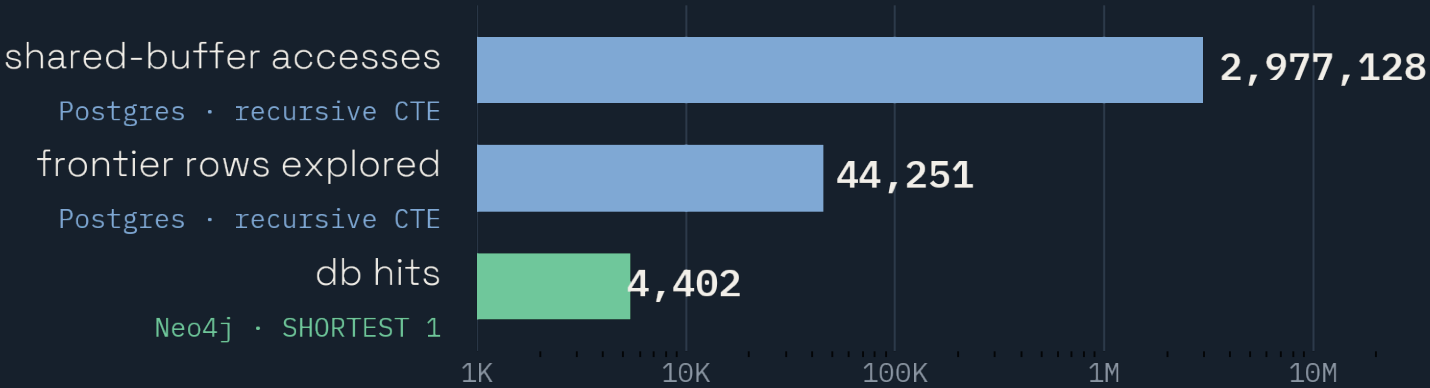
792 ms

recursive CTE (BFS)

NEO4J

14 ms

SHORTEST 1 path pattern



21 lines of SQL vs 5 of Cypher — same answer, same semantics

POSTGRESQL – RECURSIVE CTE (BFS)

```
WITH RECURSIVE bfs (player_api_id, distance) AS (  
  SELECT src, 0 FROM endpoints  
  UNION  
  SELECT pf2.player_api_id, b.distance + 1  
  FROM   bfs b  
  JOIN   soccer.mv_played_for pf1  
        ON pf1.player_api_id = b.player_api_id  
  JOIN   soccer.mv_played_for pf2  
        ON pf2.team_api_id = pf1.team_api_id  
        AND pf2.season      = pf1.season  
        AND pf2.player_api_id <> b.player_api_id  
  WHERE  b.distance < 6  
)  
SELECT MIN(distance) FROM bfs  
WHERE  player_api_id = dst;
```

21 → 5

lines of code

26 → 5

logical operators

792 → 14

milliseconds

NEO4J – SHORTEST 1 PATH PATTERN (GQL)

```
MATCH (a:Player {name: $player_a}),  
      (b:Player {name: $player_b})  
MATCH path = SHORTEST 1 (a)  
  ((x:Player)-[r1:PLAYED_FOR]->(:Team)  
  <-[r2:PLAYED_FOR]-(y:Player)  
  WHERE r1.season = r2.season){1,6}  
(b)  
RETURN length(path) / 2;
```

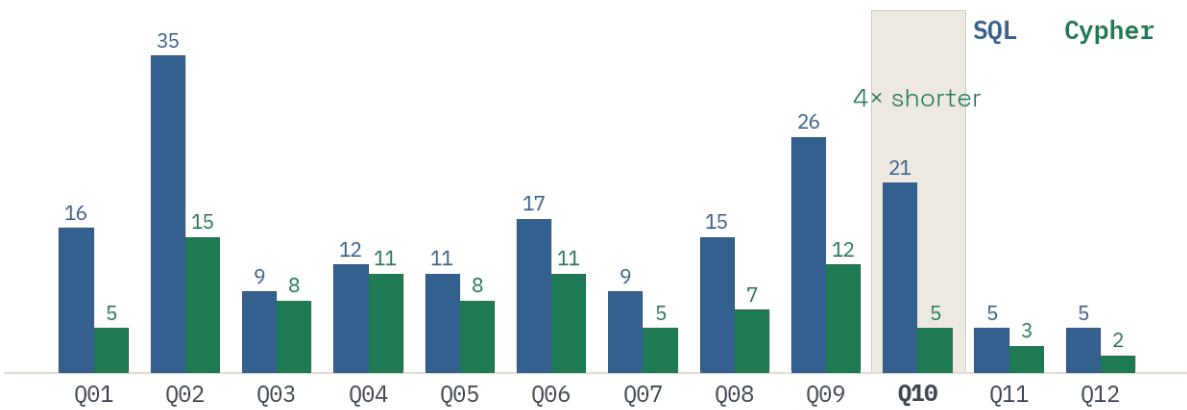
Cypher is not just faster: it's 4× shorter — and a schema change is one SET, not a migration

CODE EXPRESSIVENESS — Q10

4× fewer lines of code
21 SQL → 5 Cypher

5× fewer operators
26 SQL → 5 Cypher

LINES OF CODE PER QUERY — ALL 12, SQL VS CYPHER



SCHEMA EVOLUTION — Q12: ADD totalGoals TO EVERY MATCH

POSTGRES — 2 STEPS + LOCK

```
ALTER TABLE match
  ADD COLUMN total_goals;
UPDATE match
SET total_goals =
  home_goal + away_goal;
```

NEO4J — 1 STEP, NO DDL

```
MATCH (m:Match)
SET m.totalGoals =
  m.homeGoals
  + m.awayGoals;
```

436 ms vs 85 ms (5.1×, measured through COMMIT) — same CPU work, radically different change-management: schema-on-write vs schema-on-read.

Three takeaways from twelve queries

1

Where the depth is fixed, the relational join still wins

Two-hop teammates (Q09): same precomputed relation, B-tree joins beat the traversal 2.7× — and the gap widens with data. Light aggregations (< 25 ms): a dead heat across runs.

2

Neo4j wins where the depth is variable — by a class

Q10: 55× with identical semantics — the CTE explores 44,251 states, SHORTEST 1 touches 4,402 db hits. Q06 (2.9×) and Q07 (1.9×) follow; Q07's edge grows with data.

3

Schema flexibility is the forgotten dimension

One SET vs ALTER + UPDATE: 5.1× measured through commit. In change-heavy environments this matters more than raw milliseconds.

CAVEATS Single node, single user, default configs. Four sensitivity analyses in the report: work_mem, Q10 semantics, MVCC bloat (VACUUM protocol), scaling on 2/4/8 seasons.

Two more dimensions: ease of use and scalability

5,653

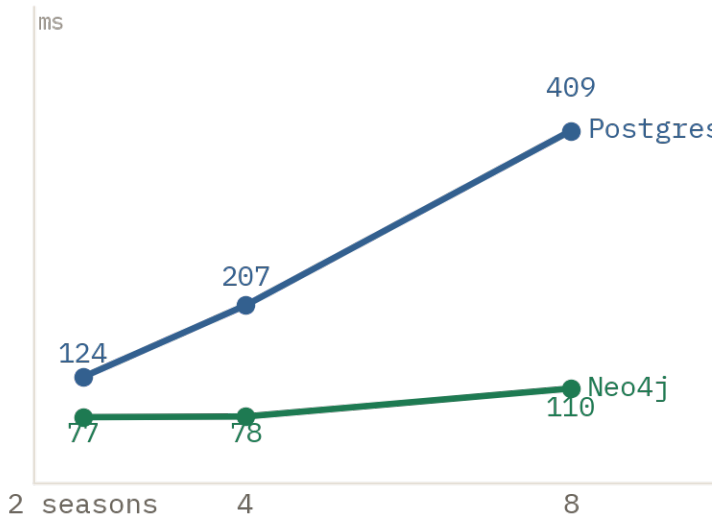
Schema-first found our bugs

- Postgres' FK check rejected 5,653 orphan player references — Neo4j would have loaded them silently.
- Running both systems side by side caught three more silent bugs: NULL semantics (Q05), path semantics (Q10), name-based grouping (Q07, Q09).
- Cypher is 2× shorter on average, 4× on the shortest path; SQL's explicit schema is the documentation.

3.3× vs 1.4×

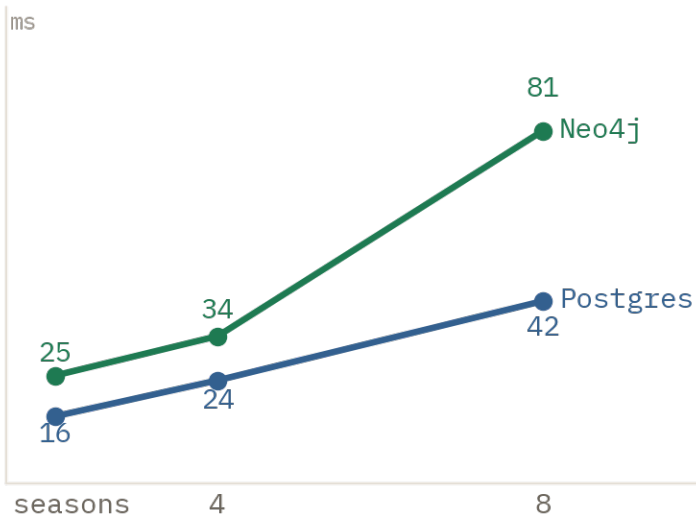
How Q07 grows with 4× the data — Postgres vs Neo4j: the graph edge widens

Q07 — players in all seasons



Postgres ×3.3, Neo4j ×1.4 — the graph edge widens

Q09 — 2-hop teammates



Postgres keeps the lead at every size

CAVEATS Index-free adjacency does not partition for free: single-node result. 10 runs per cell, medians; on Q09 Postgres keeps its lead at every size.

VERDICT

Not "which is better" — which problem do you have?

	Choose PostgreSQL	Choose Neo4j
INTEGRITY	ACID transactions and strict constraints matter	Data is naturally a network of entities and events
WORKLOAD	Reporting and OLAP-style aggregations dominate	Queries traverse paths, neighbourhoods, communities
SCHEMA	Stable and well understood	Evolves frequently and unpredictably

Thank you — live demo time →

[github.com/g1up3/DM-Project-2026 · reports/benchmark_report.md](https://github.com/g1up3/DM-Project-2026/blob/main/reports/benchmark_report.md)



scan — report, harness, plans, sensitivity analyses

Three bugs only the full load reveals — all fixed upstream

5,653 orphan player refs

Problem — XML events referenced player IDs absent from Player

Fix — nullified in transform.py

Result — 100% FK integrity, only 0.5% of attributions lost

INTEGER NULL → "673.0"

Problem — pandas float-serialised nullable integers; COPY rejected

Fix — auto-detect and cast to Int64 before CSV write

Result — clean load + player_stats.csv shrunk 38 → 25 MB

card_type CHAR(1) too narrow

Problem — the schema didn't anticipate "y2", the second-yellow code

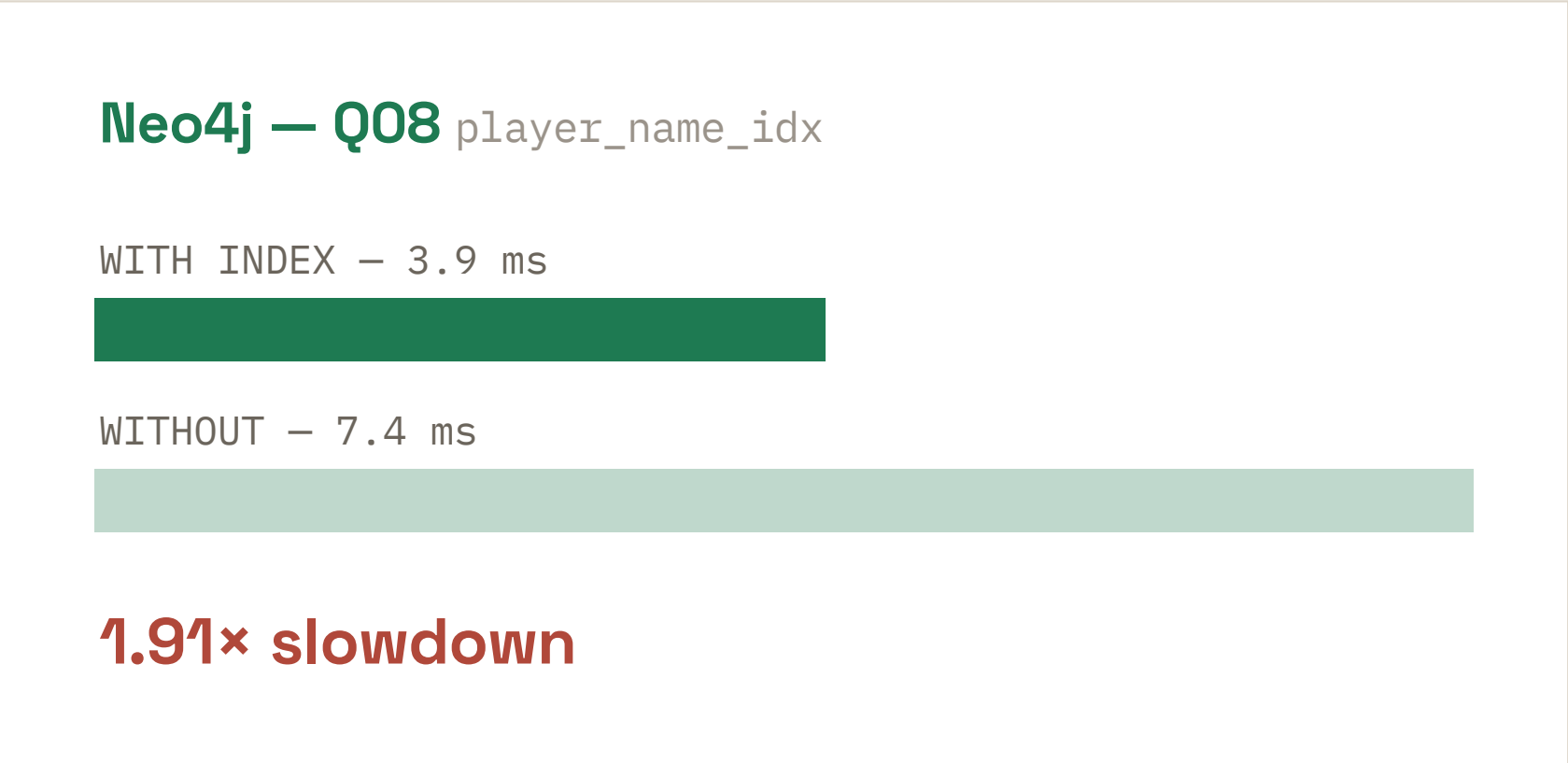
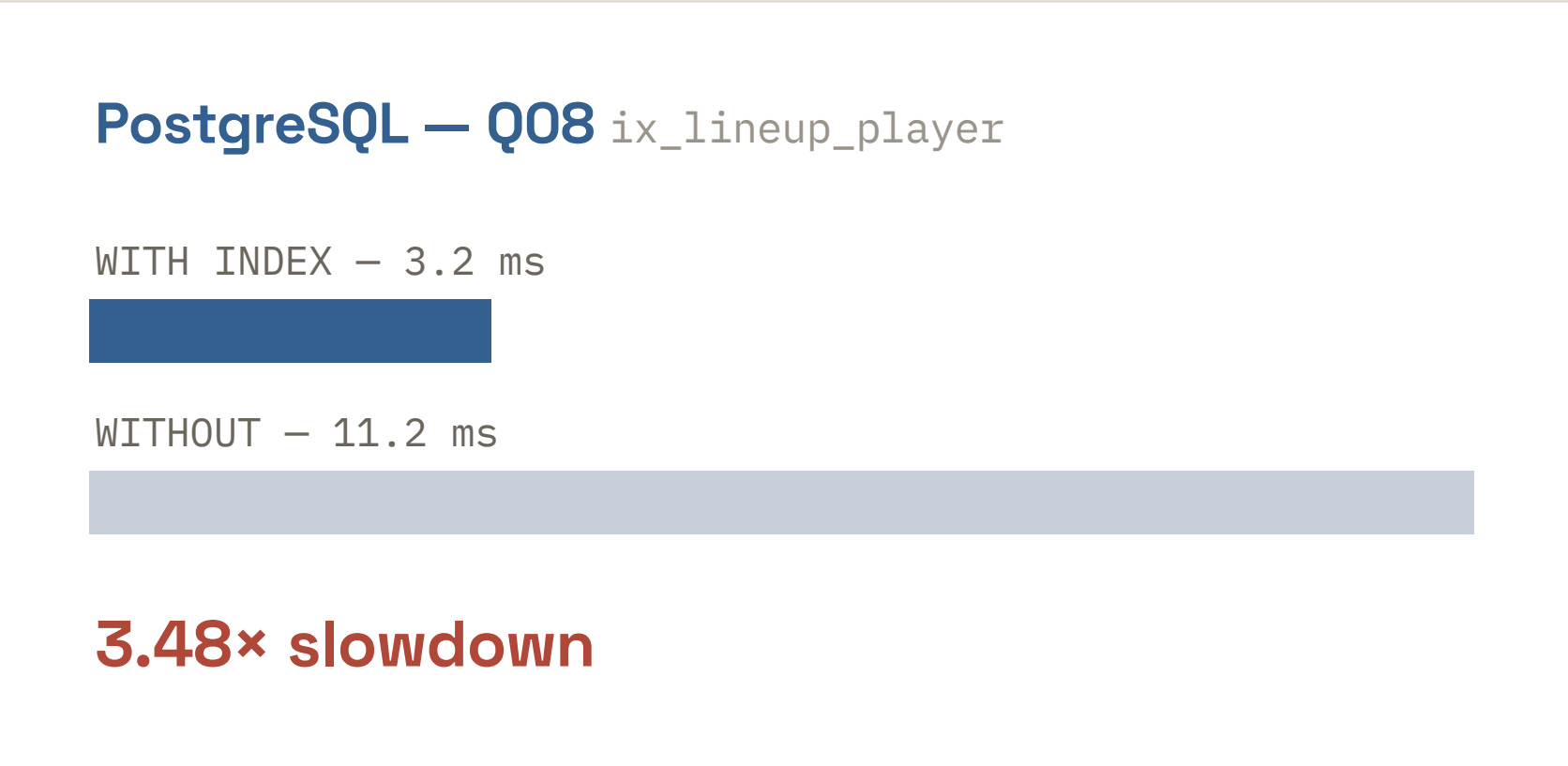
Fix — widened to VARCHAR(2)

Result — the kind of bug only the full load reveals

Each fix is small in isolation; together they make the pipeline reproducible end-to-end.

Without indexes both degrade: 3.5× on Postgres, 1.9× on Neo4j

7 index-query pairs (4 PG + 3 Neo4j) — each dropped, measured over 10 runs, restored. Shown: Q08.



What this benchmark does not measure

Single-node, single-user

Concurrency, lock contention and MVCC under load are out of scope.

No tuning

Default configs. work_mem tested on Q07: no effect. A shared_buffers / pagecache grid could shift close category-A races.

Medium scale

Measured on 2/4/8 seasons: Neo4j's aggregation edge widens with data, Postgres' fixed-depth join edge too. LDBC-SNB scale is future work.

Future work

OLTP write-heavy workload · Citus vs Neo4j Fabric · hybrid polyglot architectures.

Angles & Gutierrez (2008), ACM Comp. Surveys · Vicknair et al. (2010), ACM SE · Holzschuher & Peinl (2013), EDBT/ICDT · Erling et al. (2015), LDBC-SNB, SIGMOD · Robinson et al. (2015), Graph Databases 2nd ed.

What each query asks — and what it tests

Q01 cat. A	Top scorers by season — top-N goal scorers of one season. 3-table join + GROUP BY.
Q02 cat. A	League standings — full table of one league-season: points, goal difference. UNION ALL + aggregation.
Q03 cat. A	Goals per match by league — average goals per match in every league, one season. GROUP BY.
Q04 cat. A	Home win % by team — teams with the best home-win rate, min. N home matches. Conditional aggregation.
Q05 cat. B	Goal-assist partnerships — scorer-assister pairs that combined at least N times. Self-join on one event.
Q06 cat. B	Cards vs Real Madrid — players booked most often when playing against a given team. 4-table join.
Q07 cat. B	Players in all 8 seasons — players who appeared in every season of the dataset. COUNT DISTINCT = 8.
Q08 cat. C	Teammates of Messi 2015/16 — players who shared a lineup with X in season Y, by shared matches.
Q09 cat. C	2-hop teammates — teammates-of-teammates of X, ranked by shared team-seasons. Fixed depth, mv_played_for.
Q10 cat. C	Shortest path Messi → Pirlo — degrees of separation on the teammate graph, max 6 hops. Recursive CTE vs SHORTEST 1.
Q11 cat. D	Bulk UPDATE — lowercase the subtype of the 21,442 goal events with a known scorer. Measured to COMMIT.
Q12 cat. D	Schema evolution — add totalGoals = home + away to all 25,979 matches. ALTER + UPDATE vs one SET.

All 12 are parameterised, written twice (queries/sql · queries/cypher) and result-checked as multisets on every run.

All twelve queries – medians of 15 runs, run_20260921_162704

	QUERY	POSTGRES	NEO4J	SPEEDUP	WINNER	p	r	HOLM
Q01	Top scorers by season	10.3 ms	6.9 ms	1.51×	Neo4j	< 0.001	0.75	yes
Q02	League standings	3.7 ms	5.1 ms	1.38×	—	0.229	0.26	no
Q03	Goals per match by league	8.3 ms	5.4 ms	1.54×	Neo4j	0.028	0.47	no
Q04	Home win % by team	15.9 ms	20.7 ms	1.30×	Postgres	0.004	0.61	yes
Q05	Goal-assist partnerships	61.5 ms	38.1 ms	1.62×	Neo4j	< 0.001	0.86	yes
Q06	Cards vs Real Madrid	21.8 ms	7.6 ms	2.85×	Neo4j	< 0.001	1.0	yes
Q07	Players in all 8 seasons	400.2 ms	216.1 ms	1.85×	Neo4j	< 0.001	1.0	yes
Q08	Teammates of Messi 2015/16	7.0 ms	4.2 ms	1.66×	Neo4j	< 0.001	0.93	yes
Q09	2-hop teammates of Messi	28.9 ms	77.6 ms	2.68×	Postgres	< 0.001	1.0	yes
Q10	Shortest path Messi → Pirlo	792.1 ms	14.4 ms	54.89×	Neo4j	< 0.001	1.0	yes
Q11	Bulk UPDATE (write)	303.1 ms	195.6 ms	1.55×	Neo4j	< 0.001	0.95	yes
Q12	Schema evolution (write)	435.9 ms	85.4 ms	5.11×	Neo4j	< 0.001	1.0	yes

Mann-Whitney U two-sided · r = rank-biserial effect size · Holm = significant after Holm-Bonferroni over the 12 tests (10/12) · all 10 read queries return identical multisets

Fair comparison: the same precomputation on both sides

POSTGRES — MATERIALIZED VIEW, BUILT ONCE AT LOAD

```
CREATE MATERIALIZED VIEW mv_played_for AS
SELECT DISTINCT l.player_api_id,
               CASE WHEN l.side = 'home' THEN m.home_team_api_id
                   ELSE m.away_team_api_id END AS team_api_id,
               m.season
FROM   match_lineup l
JOIN   match m ON m.match_api_id = l.match_api_id;
CREATE INDEX ... ON mv_played_for(player_api_id);
CREATE INDEX ... ON mv_played_for(team_api_id, season);
```

NEO4J — DERIVED RELATIONSHIP, BUILT ONCE AT LOAD

```
MATCH (p:Player)-[:LINEUP_OF]->(m:Match)
      -[:HOME|AWAY]->(t:Team)
WITH  p, t, m.season AS season, count(*) AS apps
MERGE (p)-[r:PLAYED_FOR {season: season}]->(t)
SET   r.appearances = apps
```

35,002

rows in the view = 35,002 PLAYED_FOR
relationships — the two populations coincide

Q09 • Q10

the only queries that use it, on both sides; Q08
walks the raw lineups on both sides

why

without it Postgres would rebuild played_for
with a CTE on every run while Neo4j walks a
precomputed edge — asymmetric by design